Programación Segura

Conceptos de seguridad

Docente: Emilio Antonio Alarcón Santos
Contacto: zS25022557@estudiantes.uv.mx

Calidad en software

- **¿Qué es la calidad del software?**
- La calidad se refiere al grado en que un producto satisface o cumple con los requerimientos del cliente





Calidad en software

- Existe un problema de enfoque entre calidad en software y seguridad
- La seguridad suele pasar a un segundo plano en lugar de que aumente su prioridad



Calidad en software

- El software tiene tanto características de calidad **internas** como **externas**
- Donde las **externas** son aquellas en las que el usuario del sistema puede percibir, como:
  - **Usabilidad:** La facilidad para que el usuario aprenda a usar el sistema
  - **Eficiencia:** Uso mínimo de recursos, como memoria y procesador
  - **Confiabilidad:** Es la habilidad del sistema para ejecutar sus funciones bajo diversas condiciones posibles y donde este produzca la menor cantidad de errores

Calidad en software

- Otras **características externas** de calidad:
  - **Integridad:** El grado en que el sistema previene el acceso no apropiado o no autorizado al sistema y a sus datos. Esta también ligado a asegurar que los datos del usuario no se alteren de maneras impredecibles
  - **Adaptabilidad:** la extensión en que un sistema puede ser utilizado sin modificarlo
  - **Certeza:** el grado en que un sistema esta libre de errores
  - **Robustez:** es el grado en que el sistema continua funcionando ante la aparición de fallos o condiciones de estrés

Calidad en software

- Los **programadores** en cambio **se preocupan por características internas** del software así **como por características externas**
- Algunas características internas de calidad incluyen:
  - **Mantenibilidad:** facilidad de mantenimiento
  - **Flexibilidad:** que tanto puede modificarse un sistema para usos o ambientes diferentes para el que fue creado
  - **Portabilidad:** La capacidad de un programa para ejecutarse en otros sistemas operativos, plataformas o dispositivos
  - **Reusabilidad:** Utilizar fragmentos de código, componentes o módulos ya existentes para evitar la programación desde cero
  - **Legibilidad:** que tan fácil es entender el código fuente
  - **Facilidad de prueba:** que tan fácil va a ser ejecutar una prueba a partir de un proceso, componente, módulo, etc.
  - **Facilidad de entendimiento**: incluye tanto el bajo nivel como el alto nivel

Rutinas de alta calidad

Introducción

Una rutina puede ser:

- Una función (C++)
- Un método (Java)
- Un procedimiento (Pascal)
- Una macro (ensamblador)

Introducción



La seguridad en el código empieza teniendo buen código

Un código poco legible es difícil de auditar, incluyendo auditoría de seguridad

Un código poco legible es difícil de corregir o extender

Un mal código es más propenso a tener defectos de seguridad como mal manejo de memoria, condiciones de carrera o ciclos infinitos no intencionales

Rutinas de alta calidad



- Tienen un nombre descriptivo que no es demasiado corto ni demasiado largo, su nombre es un verbo
- No debe tener demasiados parámetros (aumenta el acoplamiento)
- Los parámetros deben seguir un orden lógico y consistente
- Los parámetros deben ser descriptivos
- No deberían introducir efectos colaterales
- Deben ser cohesivas (hacer solo una cosa)
- Deben ser fáciles de leer y entender (siguen una estructura lógica)

Rutinas de alta calidad



- Tienen que estar bien documentadas
- No deben manipular memoria de forma descuidada
- Deben defenderse ante datos inválidos
- No utilizan números mágicos
- Utiliza todos los parámetros que recibe

Rutinas de alta calidad



```javascript
/**
 * Calcula el precio final de un producto aplicando un descuento y un impuesto.
 *
 * @param {number} precioBase - El costo inicial del producto (debe ser mayor a 0).
 * @param {number} porcentajeDescuento - El descuento a aplicar (ej. 10 para 10%).
 * @param {number} [impuesto=0.16] - El porcentaje de impuesto (por defecto 0.16 para 16%).
 * @returns {number} El precio final facturado redondeado a dos decimales.
 */
function calcularPrecioFinal(precioBase, porcentajeDescuento, impuesto = 0.16) {
    // 1. Validación de entradas (Programación defensiva)
    if (typeof precioBase !== 'number' || precioBase <= 0) {
        throw new TypeError("El precio base debe ser un número mayor a cero.");
    }

    if (typeof porcentajeDescuento !== 'number' || porcentajeDescuento < 0 || porcentajeDescuento > 100) {
        throw new RangeError("El descuento debe ser un número entre 0 y 100.");
    }

    // 2. Procesamiento (Lógica clara y legible)
    const montoDescuento = precioBase * (porcentajeDescuento / 100);
    const precioConDescuento = precioBase - montoDescuento;

    const precioFinal = precioConDescuento * (1 + impuesto);

    // 3. Retorno preciso
    return Number(precioFinal.toFixed(2));
}
```

Buenos nombres de rutinas



- Describe lo que una rutina hace
- No utiliza verbos genéricos de apoyo (hacer, ejecutar, etc.)
- Su nombre es tan largo como hace falta
- Es buen consejo apoyarse en lo que regresa la función para nombrarla
- Utiliza un verbo seguido de un sujeto
- Se apoyan del uso de opuestos: agregar/quitar, empezar/terminar, crear/destruir, primero/último, etc.
- Siguen convenciones en sus nombres (get, set, is)



Programación defensiva

Introducción

- Adopta la misma filosofía que al manejar un carro: nunca sabes lo que los otros conductores van a hacer mejor asume lo peor
- Te proteges a ti mismo aunque no sea tu culpa
- Si a una rutina se le pasan datos malos, no deberían de hacer daño, incluso si esos datos malos son producto de una rutina mal hecha

Seguridad y sus elementos



- Aunque la programación defensiva está más orientada al tratamiento de bugs y manejo de errores esperados, también trata sobre seguridad
- En programación defensiva te proteges, por grado de importancia:
  - De ti mismo como programador (bugs)
  - De usuarios que comenten errores
  - De atacantes

Protegerse de datos de entrada inválidos



- Si recibes datos inválidos de ninguna forma se debe regresar algo, se debe generar un error
- Seguir estos consejos:
- Revisar los valores de fuentes externas: revisar si están en un rango, un formato, etc. Ten especial cuidado ante posibles ataques (inyecciones, overflows, etc.)
- Revisar los valores de todos los parámetros de rutinas: esencialmente igual al anterior
- Decidir como manejar entradas malas: dependiendo de tipo de
- mala entrada se pueden hacer diversas cosas

Técnicas para el manejo de errores



- Para manejar errores que sabes pueden ocurrir
- Dependiendo de las circunstancias puede quererse hacer algo diferente, normalmente se lanza una excepción, se termina el programa, se utiliza un valor de retorno anterior que funcionó, se despliega un mensaje de error.

Excepciones



- Las excepciones son errores que se generan en tiempo de ejecución
- Se utilizan para avisar a otras partes del código acerca de errores que no deben ser ignorados. Las excepciones se propagan hacía arriba
- Lanza una excepción solo para casos que sean realmente excepcionales. Deben ser el último recurso.
- Las excepciones mal utilizadas incrementan la complejidad
- Las excepciones reducen la encapsulación ya que las rutinas pueden requerir saber que excepciones lanza otra rutina a la que quieren llamar

Excepciones



- Siempre prefiere manejar una excepción de forma local, evita propagarlas de no ser necesario
- Evita lanzar excepciones en constructores y destructores, a menos que los manejes de manera local. De generarse una excepción es posible que el objeto no se construya bien en memoria lo que puede causar bugs raros
- Las excepciones deben manejarse en el nivel adecuado de abstracción: si se generan en una interfaz por ejemplo, deben de ser de muy alto nivel (nivel de dominio por ejemplo)
- Al generarse la excepción debería incluirse información para saber cómo se llegó a dicha excepción (error.printStackTrace())
- No uses bloques catch vacíos

Excepciones



- Crea tus propias excepciones de dominio
- Familiarízate con las excepciones de las bibliotecas que incluyes
- A veces no deben utilizarse excepciones para manejar errores, si se puede evitar su uso mejor
- Utiliza el tipo de excepción más especifico (evita usar siempre exception)
- Para seguridad: nunca dejes visible el mensaje de trace que genera una excepción, puede dar demasiada información que un atacante puede aprovechar, utiliza logs

Determinar que tanta programación defensiva dejar en producción



- Una paradoja de la programación defensiva es que durante el desarrollo quieres que los errores sean lo más obstrusivos para que sean fáciles de notar y corregir, en cambio, en producción se desea que los errores sean lo menos obstrusivos posible, para que sea posible recuperarse de ellos y continuar trabajando
- En producción sólo deberías dejar los chequeos de errores importantes
- Remueve código que revise errores triviales
- Remueve código que resulte en una terminación del programa

Determinar que tanta programación defensiva dejar en producción



- Manda los errores a un Log para ser revisados más tarde
- Revisa que los errores que mandas sean amigables
- Deja el código que permita recuperarse de errores

Ser defensivo acerca de la programación defensiva



- Demasiada programación defensiva crea sus propios problemas
- Si revisas todos los datos posibles pasados como parámetros en todas las formas concebibles en todos los lugares concebibles, tu programa será demasiado lento
- Lo que es peor, demasiada programación defensiva crea complejidad
- Además, el código defensivo es tan propenso a tener errores como cualquier otro código (más mantenimiento)
- Piensa muy bien en los apartados en que necesitas ser defensivo y establece tus prioridades

Ejemplo



- Hacer una calculadora de línea de comandos que permita resolver operaciones como suma, resta, multiplicación y división entre 2 números
- Valida entrada
- Maneja adecuadamente excepciones
- Maneja bitácoras de errores

Ejercicio



- Hacer un programa que recibe la fecha de nacimiento de una persona en 3 entradas: día, mes, año; el programa regresa la edad de la persona
- Valida entrada
- Maneja adecuadamente excepciones
- Maneja bitácoras de errores